Spentra AI Integration — Product Requirements Document (PRD)

> **Version:** 2.0
> **Date:** 2026-08-02
> **Status:** Draft — Pending Approval

Table of Contents

---

1. Overview

Product: Spentra — a full-stack expense tracking application.

Tech Stack (exact versions from codebase):

- **Backend:** Spring Boot **4.0.5**, Java 17, Maven, PostgreSQL, spring-boot-starter-webmvc, spring-boot-starter-data-jpa, spring-boot-starter-security, spring-boot-starter-validation, JJWT 0.11.5, Lombok, Google API Client 2.2.0.
- **Frontend:** Next.js **16.2.9** (App Router with route groups), React **19.2.4**, TypeScript 5, Tailwind CSS **v4** (PostCSS), lucide-react, @react-oauth/google.

Existing Project Structure:

- Backend base package: com.spentra.backend at [spentra-backend/src/main/java/com/spentra/backend/](file:///Users/salman/development/Spentra/spentra-backend/src/main/java/com/spentra/backend/)
- Frontend source: [spentra-frontend/src/](file:///Users/salman/development/Spentra/spentra-frontend/src/)
- App Router uses **route groups**: (auth)/ for login/signup, (dashboard)/ for authenticated pages.
- API client uses a centralized apiClient<T>() generic fetch wrapper in [client.ts]

(file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/client.ts).
- Each API domain has its own module file: [auth.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/auth.ts), [transactions.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/transactions.ts), [budgets.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/budgets.ts), [categories.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/categories.ts), [users.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/users.ts).
- Shared types in [types.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/types.ts).
- Provider hierarchy (root layout): ThemeProvider → SettingsProvider → AuthProvider.
- Dashboard layout ([(dashboard)/layout.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/layout.tsx)) renders <Navbar />, <MobileNav />, and a global <AddTransactionModal /> with spentra-refresh-data custom event dispatch pattern.

What this PRD covers: Integration of the **Google Gemini 1.5 Flash API** into Spentra to power three new capabilities:

1. **AI Quick-Add Assistant Page** — A dedicated conversational page where users type natural-language prompts or upload receipt photos. Gemini parses the input into a structured transaction draft. The user reviews, edits, and confirms before saving.
2. **Saved Monthly AI Insights** — AI-generated monthly financial summaries persisted in PostgreSQL. Rendered on the dashboard in a dedicated card.
3. **Liquid Glass Design System** — A visual upgrade applying frosted-glass aesthetics to all modals, bottom sheets, the new Quick-Add chip, and the AI assistant page.

---

2. Goals & Success Metrics

Goals

- Reduce manual transaction entry friction by enabling natural-language and receipt-based input.
- Provide users with actionable, AI-generated financial insights each month.
- Elevate the visual quality of the app with a premium, mobile-first liquid glass design language.

Measurable Success Metrics

Metric	Target	How to Measure
Quick-Add usage rate	≥ 30% of new transactions created via AI Quick-Add within 30 days of launch	Backend logs on POST /api/ai/parse-text and POST /api/ai/parse-receipt
Gemini parse accuracy	≥ 90% of parsed drafts require zero user edits before confirmation	Compare draft JSON vs. final confirmed JSON on the frontend
AI Insights generation	≥ 1 monthly summary generated per active user per month	Count rows in ai_summaries table per user
Mobile responsiveness	All new UI passes manual QA on 375px (iPhone SE), 390px (iPhone 15), and 412px (Pixel 7) widths	Manual visual QA
API latency (parse)	p95 < 3s for text parse, p95 < 5s for receipt image parse	Backend response time logging

---

3. Scope

In Scope

- [x] New Spring Boot service (GeminiService) wrapping the Google Gemini 1.5 Flash REST API.
- [x] POST /api/ai/parse-text — accepts a natural-language string, returns a transaction draft JSON.
- [x] POST /api/ai/parse-receipt — accepts a multipart image file, returns a transaction draft JSON.
- [x] GET /api/ai/insights — returns the saved monthly AI summary (or generates one if none exists).
- [x] POST /api/ai/insights/refresh — force-regenerates and saves a fresh monthly summary.
- [x] New PostgreSQL entity AiSummary for persisting monthly summaries.
- [x] New Next.js route /(dashboard)/ai-assistant/page.tsx — full-page conversational AI assistant interface (protected by the existing dashboard layout auth guard).
- [x] New floating **Liquid Glass Quick-Add Chip** positioned above the <MobileNav /> on mobile.
- [x] New **" Ask AI"** button in the desktop <Navbar />.
- [x] New **AI Insights Card** on the dashboard page.
- [x] Liquid glass aesthetic upgrade to the existing <Modal /> component.
- [x] Receipt photo upload support (camera + gallery) integrated into the AI assistant page.
- [x] Confirmation step: Gemini draft is shown to the user for review/edit before saving. **No auto-saving.**

Partially In Scope

- The Gemini API key is stored as an environment variable (GEMINI_API_KEY) in application.properties (same pattern as existing SPRING_DATASOURCE_URL, JWT_SECRET). No secrets management system is required for MVP.

---

4. Non-Scope (Explicitly Out of Bounds)

> [!CAUTION]
> The following items are **NOT** part of this PRD. Do NOT implement them. Do NOT add infrastructure, code, or UI for them.

- **Multi-turn conversational memory or chat history persistence.** The AI assistant page is a **single-turn parse tool**, not a chatbot with session history.
- **Streaming / Server-Sent Events (SSE)** for Gemini responses. Use standard synchronous HTTP request-response.
- **Gemini Function Calling or Tool Use** features. Use plain text prompting with JSON mode only.
- **Any Gemini model besides gemini-1.5-flash.** Do not use gemini-1.5-pro or gemini-2.0-*.
- **Smart category auto-suggestion as a standalone feature.** Category mapping happens only within the Quick-Add parsing prompt.
- **Budget alerts or anomaly detection.** Not in this phase.
- **Bank statement / CSV import.** Not in this phase.
- **Changes to existing authentication, user, category, transaction, or budget entities/endpoints** beyond what is explicitly specified.
- **New npm packages or Java dependencies** beyond what is specified in this PRD.
- **Dark mode / light mode changes** to the existing color palette. Use the existing CSS variables as-is.
- **Onboarding flows, tutorials, or tooltips** for the new features.
- **Analytics tracking, logging dashboards, or observability** tooling.
- **Rate limiting or usage quota logic** on the Spentra backend side (the existing

RateLimitingFilter is sufficient).

- **Modifying the existing dashboard insights cycle engine** (lines 186–255 in [dashboard/page.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/dashboard/page.tsx#L186-L255)). The new AI Insights Card is an **additional, separate section** on the dashboard.
- **Modifying the existing AddTransactionModal**. The AI Quick-Add flow creates transactions through the same createTransaction() API function but uses its own inline confirmation UI.

---

5. Architecture Overview

```
sequenceDiagram
    participant U as User (Mobile/Desktop)
    participant FE as Next.js Frontend
    participant BE as Spring Boot Backend
    participant G as Gemini 1.5 Flash API
    participant DB as PostgreSQL

    Note over U,FE: Flow 1: Quick-Add (Text)
    U->>FE: Types "Spent $45 on dinner at Nando's yesterday"
    FE->>BE: POST /api/ai/parse-text {prompt: "..."}
    BE->>G: Gemini API call (text prompt + system instructions)
    G-->>BE: JSON response (amount, title, category, date, type)
    BE-->>FE: TransactionDraftResponse JSON
    FE->>U: Shows draft card with "Confirm & Add" button
    U->>FE: Reviews/edits → clicks "Confirm & Add"
    FE->>BE: POST /api/expenses (existing createTransaction endpoint)
    BE->>DB: INSERT INTO expenses
    BE-->>FE: ExpenseResponse
    FE->>U: Success animation → navigate back

    Note over U,FE: Flow 2: Quick-Add (Receipt Image)
    U->>FE: Uploads receipt photo
    FE->>BE: POST /api/ai/parse-receipt (multipart/form-data)
    BE->>G: Gemini API call (image bytes + system instructions)
    G-->>BE: JSON response
    BE-->>FE: TransactionDraftResponse JSON
    FE->>U: Shows draft card with "Confirm & Add" button

    Note over U,FE: Flow 3: Monthly AI Insights
    U->>FE: Opens Dashboard
    FE->>BE: GET /api/ai/insights?month=2026-08
    BE->>DB: SELECT FROM ai_summaries WHERE user_id=? AND year_month=?
    alt Summary exists in DB
        BE-->>FE: AiSummaryResponse JSON (from DB cache)
    else No summary yet
        BE->>DB: Query user's expenses for the month
        BE->>G: Gemini API call (spending data + system prompt)
        G-->>BE: Plain text summary
        BE->>DB: INSERT INTO ai_summaries
        BE-->>FE: AiSummaryResponse JSON
    end
    FE->>U: Renders AI Insights Card on Dashboard
```

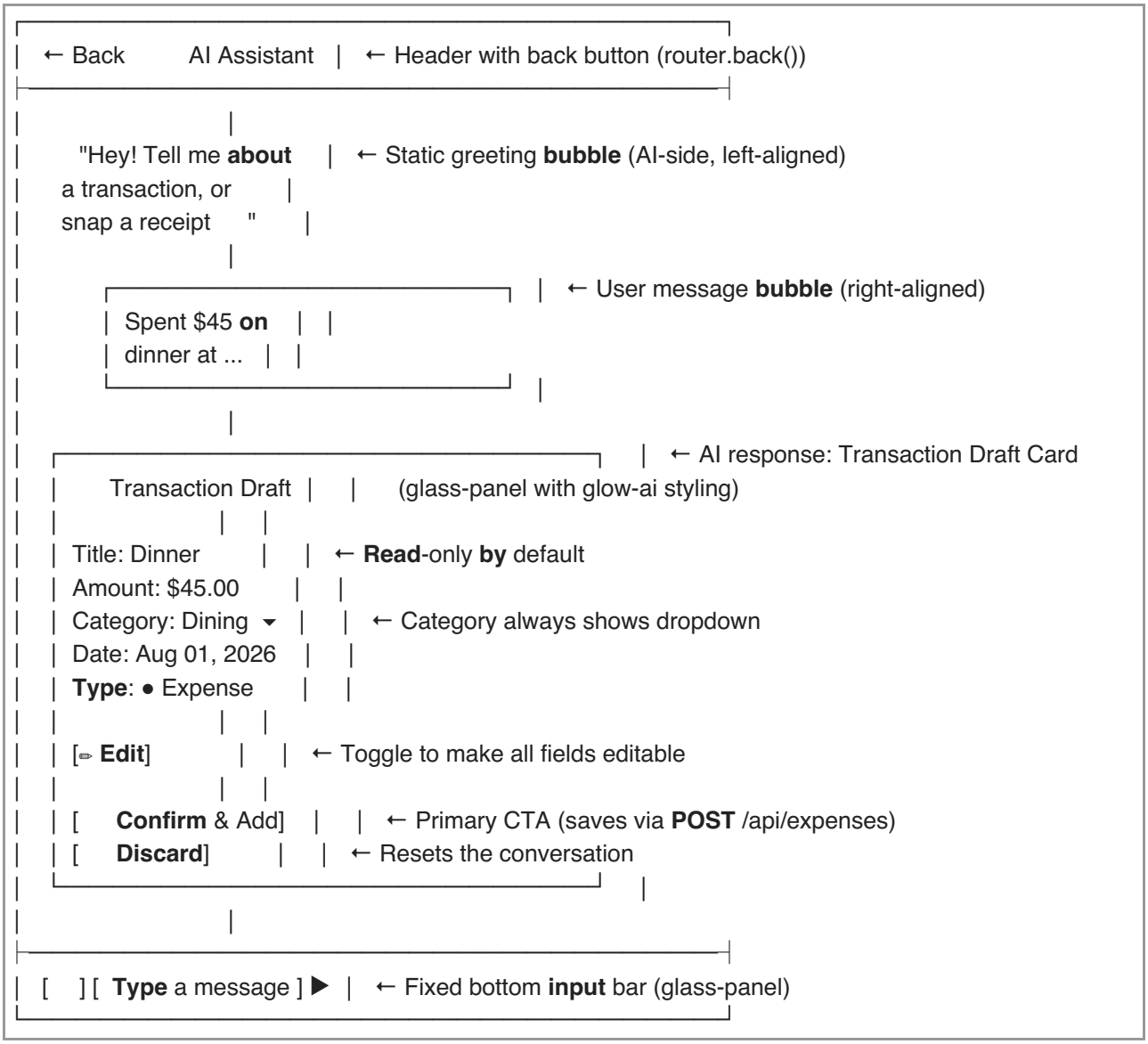
---

6. Feature 1 — AI Quick-Add Assistant Page

6.1 User Flow

1. **Entry Point (Mobile):** User sees a **floating glass chip** labeled " Ask AI" positioned directly above the existing <MobileNav /> bottom bar. The chip uses glass styling and is visible on screens below md breakpoint.

2. **Entry Point (Desktop):** An " Ask AI" button appears in the existing [Navbar](file:///Users/salman/development/Spentra/spentra-frontend/src/components/Navbar/index.tsx) component (near the right side, before the user avatar).
3. **Navigation:** Tapping/clicking navigates to /(dashboard)/ai-assistant — a new page within the existing (dashboard) route group. This means the page is automatically protected by the existing auth guard in [(dashboard)/layout.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/layout.tsx).
4. **Page Layout:** The /ai-assistant page presents a Gemini-inspired conversational interface:



5. User types a natural-language prompt (e.g., *"Paid $85 for internet bill today"*) and taps send, **OR** taps the button to upload a receipt image.
6. A **loading animation** plays (3 pulsing dots in an AI-side bubble).
7. The AI "responds" by rendering the **Transaction Draft Card** (described above).
8. User reviews the pre-filled fields. The **Category** field always shows a dropdown (populated from getCategories() via [categories.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/categories.ts)). Other fields are read-only by default — user taps **"✏ Edit"** to make them editable inline.
9. User taps **" Confirm & Add to Spentra"**:
    - Frontend calls createTransaction() from [transactions.ts]

(file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/transactions.ts) with the finalized data.
    - Dispatches window.dispatchEvent(new CustomEvent('spentra-refresh-data')) on success (same pattern as existing [dashboard layout] (file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/layout.tsx#L59)).
    - Shows a brief success checkmark animation, then navigates back via router.back().

10. User taps **" Discard"**: the draft card is removed. User can type a new prompt.

6.2 Input Constraints

- **Text prompt:** Minimum 3 characters, maximum 500 characters. Enforce on frontend (disable send button).
- **Receipt image:** Accepted formats: JPEG, PNG, WebP, HEIC. Maximum file size: 10 MB. Enforce on frontend before upload. Use accept="image/jpeg,image/png,image/webp,image/heic" and capture="environment" on the file input for mobile camera access.
- **One parse at a time:** The send button is disabled while a Gemini request is in-flight.

6.3 Category Resolution Logic (Backend)

When Gemini returns a categoryName string in the draft:

1. The **backend** fetches the user's categories (user-specific + global where user IS NULL) from CategoryRepository.
2. It performs a **case-insensitive contains match** against existing category names.
3. If a match is found → categoryId and categoryName are both set in the response.
4. If no match → categoryId is null, categoryName is returned as-is. The frontend shows the category dropdown defaulted to empty/uncategorized so the user can pick one.

---

7. Feature 2 — Saved Monthly AI Insights

7.1 User Flow

1. On the Dashboard page ([(dashboard)/dashboard/page.tsx] (file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/dashboard/page.tsx)), a new **AI Insights Card** is added to the existing grid layout — positioned in the "Bottom Section" grid (after the Spending Trends + Category Breakdown section, before the existing Budget Health + Recent Transactions row).
2. On page load, the frontend calls GET /api/ai/insights?month=YYYY-MM (using the current month from getCurrentMonth() in [utils.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/utils.ts)).
3. If a saved summary exists in the database → render it immediately.
4. If no summary exists → the backend auto-generates one via Gemini, saves it to PostgreSQL, returns it.
5. The card displays:
    - Header: " AI Monthly Insights — August 2026"
    - Body: The plain text summary (2–4 sentences of spending trends, top category, actionable tip).
    - Footer: A small **" Refresh"** button that calls POST /api/ai/insights/refresh to force-regenerate.

6. If the user has **zero transactions** for the selected month, the card shows: *"Not enough data yet. Add some transactions and check back!"* — no Gemini call is made.

7.2 Summary Content Spec

The generated summary MUST follow this structure (enforced via Gemini system prompt):

- **Line 1:** Total spending for the month.
- **Line 2:** Top spending category and its percentage of total spend.
- **Line 3:** Notable trend or observation.
- **Line 4:** One specific, actionable savings tip based on the data.

Maximum length: 500 characters. Plain text only (no markdown formatting in the stored summary). Uses the user's currency code from SettingsProvider — passed from frontend to backend as a query parameter.

---

8. Feature 3 — Liquid Glass Design System Upgrade

8.1 Design Tokens

Add the following CSS utility classes to [globals.css](file:///Users/salman/development/Spentra/spentra-frontend/src/app/globals.css) — **after** the existing .void-gradient class (around line 185) and **before** the existing @keyframes block:

```
/* ——— Liquid Glass Design System
——————————————————————————————————————————————————————————————————————— */

.glass-panel {
  background: rgba(255, 255, 255, 0.06);
  backdrop-filter: blur(20px) saturate(180%);
  -webkit-backdrop-filter: blur(20px) saturate(180%);
  border: 1px solid rgba(255, 255, 255, 0.12);
  box-shadow:
    0 8px 32px rgba(0, 0, 0, 0.08),
    inset 0 1px 0 rgba(255, 255, 255, 0.08);
}

:root .glass-panel {
  background: rgba(255, 255, 255, 0.55);
  border: 1px solid rgba(255, 255, 255, 0.60);
  box-shadow:
    0 8px 32px rgba(0, 0, 0, 0.06),
    inset 0 1px 0 rgba(255, 255, 255, 0.50);
}

.glass-chip {
  background: rgba(255, 255, 255, 0.10);
  backdrop-filter: blur(16px) saturate(180%);
  -webkit-backdrop-filter: blur(16px) saturate(180%);
  border: 1px solid rgba(255, 255, 255, 0.15);
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.06);
}

:root .glass-chip {
  background: rgba(255, 255, 255, 0.60);
  border: 1px solid rgba(255, 255, 255, 0.70);
}

.glow-ai {
  box-shadow:
    0 0 20px rgba(112, 115, 255, 0.15),
    0 0 60px rgba(112, 115, 255, 0.05);
}
```

> [!NOTE]
> These glass styles respect the existing theme system: .dark class is on <html> (default), :root styles apply when .light class is set. The existing --tertiary: #4a4bd7 / #7073ff (dark) color is used for the AI glow via rgba(112, 115, 255, ...) which maps to #7073FF.

8.2 Components to Upgrade

Existing [Modal/index.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/components/Modal/index.tsx)

Current panel class list (lines 131–141):

```
'bg-surface-container-lowest',
'rounded-t-[2.5rem] sm:rounded-[1.5rem] shadow-2xl',
```

Replace with:

```
'glass-panel',
'rounded-t-[2.5rem] sm:rounded-[1.5rem]',
```

> Remove shadow-2xl since glass-panel includes its own box-shadow. Remove bg-surface-container-lowest since glass-panel provides its own background.

> [!IMPORTANT]
> All modals across the app (AddTransactionModal, SetBudgetModal, ManageCategoriesModal, Month Summary Modal) automatically inherit glass styling since they all use the shared <Modal /> component.

Existing [MobileNav/index.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/components/MobileNav/index.tsx)

Already uses glass styling (bg-surface/30 backdrop-blur-[24px] backdrop-saturate-[180%]). **No changes needed.**

8.3 New UI Elements Requiring Glass

Element	Glass Class	Location
Floating Quick-Add Chip	glass-chip glow-ai	Above MobileNav in (dashboard)/layout.tsx
AI Assistant — bottom input bar	glass-panel	Fixed at bottom of /ai-assistant page
AI Assistant — draft card	glass-panel glow-ai	Inline in conversation area
Dashboard — AI Insights Card	glass-panel glow-ai	New section in dashboard/page.tsx

---

9. Backend Implementation Spec

9.1 New Dependency

> [!IMPORTANT]
> **Do NOT add any Maven dependency for Gemini SDK.** Use Spring's existing RestTemplate (available via spring-boot-starter-webmvc) to call the Gemini REST API directly at https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent. This keeps the dependency footprint minimal.

9.2 Configuration

Add to [application.properties](file:///Users/salman/development/Spentra/spentra-backend/src/main/resources/application.properties):

```
# Gemini AI
gemini.api.key=${GEMINI_API_KEY}
gemini.model=gemini-1.5-flash
gemini.api.url=https://generativelanguage.googleapis.com/v1beta/models
```

[NEW] config/GeminiConfig.java

Location: com.spentra.backend.config.GeminiConfig

```
@Configuration
public class GeminiConfig {
    @Value("${gemini.api.key}")
    private String apiKey;

    @Value("${gemini.model}")
    private String model;

    @Value("${gemini.api.url}")
    private String apiUrl;

    @Bean
    public RestTemplate geminiRestTemplate() {
        return new RestTemplate();
    }

    // Getters for apiKey, model, apiUrl
}
```

9.3 New Entity

[NEW] model/entity/AiSummary.java

Location: com.spentra.backend.model.entity.AiSummary

```java
@Getter @Setter
@Entity
@Table(name = "ai_summaries", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"user_id", "year_month"})
})
public class AiSummary {
    @Id
    @GeneratedValue
    private UUID id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(name = "year_month", nullable = false, length = 7)
    private String yearMonth;  // "YYYY-MM" format

    @Column(name = "summary_text", nullable = false, columnDefinition = "TEXT")
    private String summaryText;

    @Column(name = "total_spent")
    private Double totalSpent;

    @Column(name = "top_category")
    private String topCategory;

    @Column(name = "generated_at", nullable = false)
    private LocalDateTime generatedAt;

    @PrePersist
    void prePersist() {
        this.generatedAt = LocalDateTime.now();
    }
}
```

9.4 New Repository

[NEW] repository/AiSummaryRepository.java

```java
public interface AiSummaryRepository extends JpaRepository<AiSummary, UUID> {
    Optional<AiSummary> findByUserIdAndYearMonth(UUID userId, String yearMonth);
    void deleteByUserIdAndYearMonth(UUID userId, String yearMonth);
}
```

9.5 New DTOs

All in package com.spentra.backend.model.dto.ai:

[NEW] QuickAddRequest.java

```
@Getter @Setter
public class QuickAddRequest {
    @NotBlank
    @Size(min = 3, max = 500)
    private String prompt;
}
```

[NEW] TransactionDraftResponse.java

```
@Getter @Setter @AllArgsConstructor @NoArgsConstructor
public class TransactionDraftResponse {
    private String title;
    private Double amount;
    private String type;         // "EXPENSE" or "CREDIT"
    private String categoryName;  // Suggested name from Gemini
    private UUID categoryId;      // Resolved UUID or null
    private String transactionDate; // "YYYY-MM-DD"
    private String confidence;    // "HIGH", "MEDIUM", or "LOW"
}
```

[NEW] AiSummaryResponse.java

```
@Getter @Setter @AllArgsConstructor @NoArgsConstructor
public class AiSummaryResponse {
    private UUID id;
    private String yearMonth;
    private String summaryText;
    private Double totalSpent;
    private String topCategory;
    private LocalDateTime generatedAt;
}
```

9.6 New Service

[NEW] service/GeminiService.java

Location: com.spentra.backend.service.GeminiService

Injected dependencies: GeminiConfig, CategoryRepository, ExpenseRepository, AiSummaryRepository, UserRepository, RestTemplate

Methods:

1. **TransactionDraftResponse parseText(String prompt, UUID userId)**
    - Fetch user's categories from CategoryRepository.findByUserIdOrUserIsNull(userId).
    - Build Gemini API request body with system prompt (Section 11.1) + user content.
    - Call POST {apiUrl}/{model}:generateContent?key={apiKey} via RestTemplate.
    - Parse candidates[0].content.parts[0].text from response.
    - Deserialize the text as JSON into internal fields.
    - Normalize: amount = Math.abs(amount).
    - Resolve categoryId by matching categoryName against user's categories (case-insensitive contains).

- Return TransactionDraftResponse. **Do NOT save to database.**

2. **TransactionDraftResponse parseReceipt(MultipartFile image, UUID userId)**
    - Validate file size ≤ 10 MB, MIME type ∈ {image/jpeg, image/png, image/webp, image/heic}. Throw ApiRequestException(HttpStatus.BAD_REQUEST, ...) on failure.
    - Base64-encode the image bytes.
    - Build Gemini request with inlineData part (base64 + mimeType) + system prompt (Section 11.2).
    - Same parsing and category resolution as parseText.
    - Return TransactionDraftResponse.
3. **AiSummaryResponse getOrGenerateInsights(UUID userId, String yearMonth, String currencyCode)**
    - Check AiSummaryRepository.findByUserIdAndYearMonth(userId, yearMonth).
    - If present → map entity to AiSummaryResponse and return.
    - If absent → call generateAndSaveInsights(userId, yearMonth, currencyCode).
4. **AiSummaryResponse refreshInsights(UUID userId, String yearMonth, String currencyCode)**
    - Delete existing: AiSummaryRepository.deleteByUserIdAndYearMonth(userId, yearMonth).
    - Call generateAndSaveInsights(userId, yearMonth, currencyCode).
5. **private AiSummaryResponse generateAndSaveInsights(UUID userId, String yearMonth, String currencyCode)**
    - Parse yearMonth → compute startDate (1st of month) and endDate (last day of month).
    - Query: ExpenseRepository.findByUserIdAndTypeAndTransactionDateBetween(userId, TransactionType.EXPENSE, startDate, endDate).
    - If zero results → return response with summaryText = "Not enough data yet. Add some transactions and check back!", totalSpent = 0.0, topCategory = null. **Do NOT call Gemini.**
    - Aggregate: total spent, spending by category name, transaction count, top category.
    - Build Gemini prompt (Section 11.3) with aggregated data and currencyCode.
    - Call Gemini API, extract text response.
    - Create & save AiSummary entity.
    - Return AiSummaryResponse.

9.7 New Controller

[NEW] controller/AiController.java

```java
@RestController
@RequestMapping("/api/ai")
@RequiredArgsConstructor
public class AiController {

    private final GeminiService geminiService;

    @PostMapping("/parse-text")
    public ResponseEntity<TransactionDraftResponse> parseText(
            @RequestBody @Valid QuickAddRequest request) {
        UUID userId = getCurrentUserId(); // from SecurityContextHolder
        return ResponseEntity.ok(geminiService.parseText(request.getPrompt(), userId));
    }

    @PostMapping("/parse-receipt")
    public ResponseEntity<TransactionDraftResponse> parseReceipt(
            @RequestParam("file") MultipartFile file) {
        UUID userId = getCurrentUserId();
        return ResponseEntity.ok(geminiService.parseReceipt(file, userId));
    }

    @GetMapping("/insights")
    public ResponseEntity<AiSummaryResponse> getInsights(
            @RequestParam(value = "month", required = false) String month,
            @RequestParam(value = "currency", defaultValue = "INR") String currency) {
        UUID userId = getCurrentUserId();
        String yearMonth = (month != null) ? month : YearMonth.now().toString();
        return ResponseEntity.ok(geminiService.getOrGenerateInsights(userId, yearMonth, currency));
    }

    @PostMapping("/insights/refresh")
    public ResponseEntity<AiSummaryResponse> refreshInsights(
            @RequestParam(value = "month", required = false) String month,
            @RequestParam(value = "currency", defaultValue = "INR") String currency) {
        UUID userId = getCurrentUserId();
        String yearMonth = (month != null) ? month : YearMonth.now().toString();
        return ResponseEntity.ok(geminiService.refreshInsights(userId, yearMonth, currency));
    }

    private UUID getCurrentUserId() {
        String principal = (String) SecurityContextHolder.getContext()
                .getAuthentication().getPrincipal();
        return UUID.fromString(principal);
    }
}
```

All /api/ai/** endpoints automatically require JWT authentication — the existing SecurityConfig only permits /api/auth/**, /api/health, and /health without auth. No config changes needed.

9.8 Addition to Existing Repository

Add to [ExpenseRepository.java](file:///Users/salman/development/Spentra/spentra-backend/src/main/java/com/spentra/backend/repository/ExpenseRepository.java):

```
List<Expense> findByUserIdAndTypeAndTransactionDateBetween(
    UUID userId, TransactionType type, LocalDate startDate, LocalDate endDate);
```

---

10. Frontend Implementation Spec

10.1 New API Module

[NEW] lib/api/ai.ts

```ts
/**
 * @file ai.ts
 * @description AI assistant API operations via the Spentra backend.
 */

import { apiClient } from './client';
import type { TransactionDraft, AiSummaryResponse } from './types';

/** Parse a natural-language prompt into a transaction draft. */
export async function parseTextPrompt(prompt: string): Promise<TransactionDraft> {
  return apiClient<TransactionDraft>('/api/ai/parse-text', {
    method: 'POST',
    body: JSON.stringify({ prompt }),
  });
}

/** Parse a receipt image into a transaction draft. */
export async function parseReceiptImage(file: File): Promise<TransactionDraft> {
  const formData = new FormData();
  formData.append('file', file);

  // Must NOT set Content-Type header — browser sets multipart boundary automatically
  return apiClient<TransactionDraft>('/api/ai/parse-receipt', {
    method: 'POST',
    body: formData,
    headers: {}, // Override to prevent Content-Type: application/json
  });
}

/** Fetch saved AI monthly insights (auto-generates if none exist). */
export async function getAiInsights(month: string, currency: string): Promise<AiSummaryResponse>
{
  return apiClient<AiSummaryResponse>(`/api/ai/insights?month=${month}&currency=${currency}`);
}

/** Force-regenerate AI monthly insights. */
export async function refreshAiInsights(month: string, currency: string):
Promise<AiSummaryResponse> {
  return apiClient<AiSummaryResponse>(`/api/ai/insights/refresh?month=${month}&currency=${currency}`,
{
    method: 'POST',
  });
}
```

10.2 Modification to API Client

[MODIFY] [client.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/client.ts)

The current apiClient always sets 'Content-Type': 'application/json' (line 72). For multipart/form-data requests (receipt upload), this header must NOT be set. Modify the header construction:

Current (lines 71–74):

```
const headers: Record<string, string> = {
    'Content-Type': 'application/json',
    ...(options.headers as Record<string, string> | undefined),
};
```

Replace with:

```
const headers: Record<string, string> = {
    ...(options.body instanceof FormData ? {} : { 'Content-Type': 'application/json' }),
    ...(options.headers as Record<string, string> | undefined),
};
```

> This automatically detects FormData bodies and omits the JSON content type, allowing the browser to set the correct multipart/form-data boundary.

10.3 New Type Definitions

Add to [types.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/types.ts):

```
/* ——— AI Assistant ——————————————————————————————————————————————————————— */

/** Parsed transaction draft returned by the AI parsing endpoints */
export interface TransactionDraft {
  title: string;
  amount: number;
  type: 'EXPENSE' | 'CREDIT';
  categoryName: string | null;
  categoryId: string | null;
  transactionDate: string; // YYYY-MM-DD
  confidence: 'HIGH' | 'MEDIUM' | 'LOW';
}

/** Saved monthly AI summary */
export interface AiSummaryResponse {
  id: string;
  yearMonth: string;
  summaryText: string;
  totalSpent: number;
  topCategory: string | null;
  generatedAt: string;
}
```

10.4 New Components & Pages

[NEW] components/LiquidQuickAddChip/index.tsx

- **Mobile (below md):** Fixed-position glass pill floating above the MobileNav.
    - Position: fixed bottom-[calc(env(safe-area-inset-bottom)+5.5rem)] left-1/2 -translate-x-1/2

z-50 md:hidden
    - Styling: glass-chip glow-ai rounded-full px-4 py-2.5 flex items-center gap-2 cursor-pointer
    - Content: <Sparkles className="w-4 h-4 text-tertiary" /> + text "Ask AI" in text-sm font-semibold text-on-surface
    - Hover: hover:scale-105 active:scale-95 transition-all duration-200
    - On click: router.push('/ai-assistant')

- **Desktop (md+):** Not rendered. Desktop entry point is in Navbar.
- **Visibility:** Same conditional rendering as <MobileNav /> — hidden on auth pages, visible only when authenticated. Render in [(dashboard)/layout.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/layout.tsx) alongside <MobileNav />.

[NEW] app/(dashboard)/ai-assistant/page.tsx

Full-page AI assistant within the (dashboard) route group. Described in detail in Section 6.1.

Key implementation notes:

- Uses useRouter() from next/navigation for router.back() navigation.
- Uses getCategories() from [categories.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/categories.ts) to populate category dropdown in draft card.
- Uses createTransaction() from [transactions.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/transactions.ts) to save confirmed transactions.
- Uses useSettings() from [SettingsProvider](file:///Users/salman/development/Spentra/spentra-frontend/src/providers/SettingsProvider.tsx) for currency formatting.
- Dispatches new CustomEvent('spentra-refresh-data') on successful transaction creation.

[NEW] features/ai/AiInsightsCard.tsx

- Uses getAiInsights() and refreshAiInsights() from the new lib/api/ai.ts.
- Uses useSettings() for currency code.
- Uses getCurrentMonth() from [utils.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/utils.ts) for the default month.
- Listens to 'spentra-refresh-data' event to refetch insights when new transactions are added.
- Styling: glass-panel glow-ai rounded-[1.5rem] p-6 sm:p-8
- Header: " AI Monthly Insights — {monthName} {year}" using lucide Sparkles icon.
- Body: Plain text summaryText.
- Footer: Small refresh button with RefreshCw icon. Shows loading spinner while refreshing.
- Empty state: Muted text, no refresh button.

10.5 Modifications to Existing Files

File	Change
[Modal/index.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/components/Modal/index.tsx)	Replace bg-surface-container-lowest + shadow-2xl with glass-panel on the panel div
[(dashboard)/layout.tsx](file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/layout.tsx)	Add <LiquidQuickAddChip /> after <MobileNav />
[Navbar/index.tsx](file:///Users/salman/development/Spentra/spentra-	Add " Ask AI" button/link near the right

frontend/src/components/Navbar/index.tsx)	side, navigates to /ai-assistant
[(dashboard)/dashboard/page.tsx] (file:///Users/salman/development/Spentra/spentra-frontend/src/app/(dashboard)/dashboard/page.tsx)	Add <AiInsightsCard /> as a new md:col-span-2 grid item in the "Bottom Section" (around line 530)
[globals.css] (file:///Users/salman/development/Spentra/spentra-frontend/src/app/globals.css)	Add .glass-panel, .glass-chip, .glow-ai utility classes
[client.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/client.ts)	Add FormData detection for multipart Content-Type handling
[types.ts](file:///Users/salman/development/Spentra/spentra-frontend/src/lib/api/types.ts)	Add TransactionDraft and AiSummaryResponse interfaces

---

11. Gemini Prompt Engineering

> [!IMPORTANT]
> All prompts use **separate system/user roles** in the Gemini API request body. The user's raw input is NEVER concatenated into the system instructions — it is always placed in the user role content to prevent prompt injection.

11.1 System Prompt for Text Parsing

```
You are a transaction parser for a personal finance app called Spentra.
The user will describe a financial transaction in natural language.
Extract the following fields and return ONLY valid JSON (no markdown, no code fences, no explanation):

{
  "title": "Short descriptive title for the transaction",
  "amount": <number, always positive>,
  "type": "EXPENSE" or "CREDIT",
  "categoryName": "Best matching category from this list: [CATEGORY_LIST]",
  "transactionDate": "YYYY-MM-DD",
  "confidence": "HIGH" or "MEDIUM" or "LOW"
}

Rules:
- "type" is "CREDIT" only if the user explicitly mentions receiving money, salary, refund, income, or transfer in. Default to "EXPENSE".
- "transactionDate": If the user says "today", use [TODAY_DATE]. If "yesterday", use [YESTERDAY_DATE]. If no date mentioned, use [TODAY_DATE].
- "categoryName": Pick the closest match from the provided category list. If nothing fits, use "Uncategorized".
- "confidence": Use "HIGH" if all fields are clearly stated, "MEDIUM" if you inferred 1-2 fields, "LOW" if the input is vague or ambiguous.
- "amount": Must be a positive number. If the user doesn't specify a currency, assume the amount as given.
- Do NOT include any text outside the JSON object. No markdown fences. No explanation.
```

Dynamic placeholders replaced at runtime by GeminiService:

- [CATEGORY_LIST] → Comma-separated category names from DB (e.g., "Food & Dining, Transport, Shopping, Utilities, Entertainment")
- [TODAY_DATE] → LocalDate.now().toString() (e.g., "2026-08-02")
- [YESTERDAY_DATE] → LocalDate.now().minusDays(1).toString()

11.2 System Prompt for Receipt Parsing

```
You are a receipt parser for a personal finance app called Spentra.
Analyze the provided receipt image and extract transaction details.
Return ONLY valid JSON (no markdown, no code fences, no explanation):

{
  "title": "Merchant or store name from the receipt",
  "amount": <total amount as a positive number>,
  "type": "EXPENSE",
  "categoryName": "Best matching category from this list: [CATEGORY_LIST]",
  "transactionDate": "YYYY-MM-DD",
  "confidence": "HIGH" or "MEDIUM" or "LOW"
}

Rules:
- Extract the TOTAL amount (not subtotal, not tax-only). Use the final/grand total.
- "type" is always "EXPENSE" for receipts.
- "transactionDate": Extract from the receipt. If not readable, use [TODAY_DATE].
- "confidence": "HIGH" if text is clearly readable, "MEDIUM" if partially, "LOW" if blurry/ambiguous.
- Do NOT include any text outside the JSON object.
```

11.3 System Prompt for Monthly Insights

```
You are a financial advisor AI for a personal finance app called Spentra.
Analyze the following monthly spending data and provide exactly 4 concise sentences.

User's spending for [YEAR_MONTH]:
- Total spent: [CURRENCY_SYMBOL][TOTAL_SPENT]
- Breakdown by category:
[CATEGORY_BREAKDOWN]
- Total transactions: [TX_COUNT]

Return ONLY 4 plain text sentences (no markdown, no bullets, no headers, no numbering):
Sentence 1: Total spending summary for the month.
Sentence 2: Top spending category and its share of total spend.
Sentence 3: A notable trend or observation.
Sentence 4: One specific, actionable savings tip based on the data.

Keep the total response under 500 characters. Be specific to the numbers. No generic advice.
```

Dynamic placeholders:

- [YEAR_MONTH] → e.g., "August 2026"
- [CURRENCY_SYMBOL] → e.g., "₹" or "{{content}}quot; based on currencyCode parameter
- [TOTAL_SPENT] → Aggregated total
- [CATEGORY_BREAKDOWN] → e.g., "Food & Dining: ₹12,500 (42%), Transport: ₹8,000 (27%),

Shopping: ₹5,200 (17%)"
- [TX_COUNT] → Count of expense transactions

---

12. Error Handling & Edge Cases

Scenario	Backend Behavior	Frontend Behavior
Gemini returns non-JSON or malformed response	Throw ApiRequestException(BAD_REQUEST, "Could not parse AI response. Please try again.")	Show error message in AI chat bubble. Allow retry.
Gemini API timeout (>10s)	Configure RestTemplate SimpleClientHttpRequestFactory with 10s timeout. Throw ApiRequestException(GATEWAY_TIMEOUT, "AI service timed out.")	Show error in chat bubble. Allow retry.
Gemini API key invalid / quota exceeded	Throw ApiRequestException(SERVICE_UNAVAILABLE, "AI service temporarily unavailable.")	Show error toast. Suggest manual entry.
Receipt image too large (>10 MB)	Throw ApiRequestException(BAD_REQUEST, "Image file size must be under 10 MB.")	Validate on frontend before upload. Show inline error.
Receipt image unsupported format	Throw ApiRequestException(BAD_REQUEST, "Unsupported image format. Use JPEG, PNG, WebP, or HEIC.")	accept attribute on file input restricts selection.
Zero transactions for insights month	Return AiSummaryResponse with empty-state text. Do NOT call Gemini. Do NOT save to DB.	Show muted empty state card. Hide refresh button.
Text prompt < 3 chars	400 via @Valid @Size on QuickAddRequest	Disable send button when input < 3 chars.
Category resolution: no match	Return draft with categoryId = null	Category dropdown defaults to empty; user must select.
Gemini returns negative amount	Backend normalizes: Math.abs(amount)	N/A
Network error calling Gemini	Catch and throw ApiRequestException(BAD_GATEWAY, "Could not reach AI service.")	Show error. Allow retry.

---

13. Security Constraints

> [!WARNING]
> Non-negotiable requirements.

1. **The Gemini API key MUST NEVER be exposed to the frontend.** It exists only in backend application.properties via ${GEMINI_API_KEY}.
2. **All /api/ai/** endpoints require JWT authentication** — enforced by existing SecurityConfig (only /api/auth/**, /api/health, /health are permitted without auth).
3. **User isolation:** GeminiService MUST always scope category and transaction queries to the authenticated user's userId (extracted from SecurityContextHolder, same pattern as

ExpenseService).

4. **Prompt injection prevention:** User text input is placed in the user role content in the Gemini API request body. It is NEVER concatenated into the system role instructions.
5. **Server-side validation:** Image uploads MUST be validated for MIME type and file size on the backend, not just the frontend.

---

14. Testing & Verification Plan

14.1 Backend Unit Tests

Test	Description
GeminiServiceTest.parseText_validPrompt_returnsDraft	Mock RestTemplate, verify JSON parsing and category resolution
GeminiServiceTest.parseText_malformedResponse_throwsException	Verify error handling for non-JSON Gemini output
GeminiServiceTest.parseReceipt_oversizedFile_throwsException	Verify 10 MB file size limit
GeminiServiceTest.parseReceipt_invalidMimeType_throwsException	Verify MIME type validation
GeminiServiceTest.getOrGenerateInsights_existingSummary_returnsFromDb	Verify DB cache hit (no Gemini call)
GeminiServiceTest.getOrGenerateInsights_noTransactions_returnsEmptyState	Verify no Gemini call when zero transactions
GeminiServiceTest.refreshInsights_deletesOldAndGeneratesNew	Verify delete + regenerate flow
AiControllerTest.parseText_unauthenticated_returns401	Verify JWT requirement
AiControllerTest.parseText_validRequest_returns200	Integration test with mocked Gemini

14.2 Frontend Manual QA Checklist

- [ ] Floating Quick-Add chip visible on mobile (< md), hidden on desktop
- [ ] Desktop " Ask AI" button visible in Navbar
- [ ] Both entry points navigate to /ai-assistant
- [ ] AI assistant page shows static greeting bubble
- [ ] Text input → send → loading dots → draft card appears
- [ ] Draft card shows all parsed fields correctly
- [ ] Category dropdown populates from user's categories
- [ ] "✏ Edit" toggle makes fields editable
- [ ] " Confirm & Add" creates transaction → success animation → navigates back
- [ ] Dashboard data refreshes after Quick-Add creation (spentra-refresh-data event)
- [ ] " Discard" clears draft and resets input
- [ ] Receipt upload ( ) triggers file picker with correct accept types
- [ ] Receipt parsing returns draft card
- [ ] AI Insights card loads on dashboard page
- [ ] Insights card shows stored summary text
- [ ] Refresh button regenerates summary (loading state visible)
- [ ] Empty state shown when no transactions for month
- [ ] All existing modals (Add Transaction, Edit Transaction, Set Budget, Manage Categories, Month Summary) now show liquid glass styling

- [ ] Glass styling works correctly in both dark and light themes
- [ ] UI renders correctly on 375px, 390px, and 412px screen widths
- [ ] Back button on AI assistant page works (router.back())

---

15. Implementation Order

Execute in this exact sequence. Do not skip steps.

```
graph TD
   S1["Step 1: globals.css — Add liquid glass CSS tokens"] --> S2["Step 2: Modal — Replace bg with glass-panel"]
   S2 --> S3["Step 3: Backend — application.properties + GeminiConfig"]
   S3 --> S4["Step 4: Backend — AiSummary entity + AiSummaryRepository"]
   S4 --> S5["Step 5: Backend — New DTOs (QuickAddRequest, TransactionDraftResponse, AiSummaryResponse)"]
   S5 --> S6["Step 6: Backend — Add query to ExpenseRepository"]
   S6 --> S7["Step 7: Backend — GeminiService (all 5 methods)"]
   S7 --> S8["Step 8: Backend — AiController (all 4 endpoints)"]
   S8 --> S9["Step 9: Backend — Unit tests for GeminiService + AiController"]
   S9 --> S10["Step 10: Frontend — client.ts FormData fix"]
   S10 --> S11["Step 11: Frontend — types.ts + new ai.ts API module"]
   S11 --> S12["Step 12: Frontend — LiquidQuickAddChip component"]
   S12 --> S13["Step 13: Frontend — /ai-assistant page"]
   S13 --> S14["Step 14: Frontend — AiInsightsCard component"]
   S14 --> S15["Step 15: Frontend — Wire into layout.tsx + dashboard/page.tsx + Navbar"]
   S15 --> S16["Step 16: Visual QA — Mobile + Desktop + Theme toggle"]
   S16 --> S17["Step 17: End-to-end QA — Full flows with live Gemini API"]
```

> [!IMPORTANT]
> Each step should be a distinct, testable unit of work. Backend steps (3–9) should be completed and verified before starting frontend steps (10–15). Within each group, follow the order strictly — later steps depend on earlier ones.